DS604 Introduction to Data Management, Autumn 2026

Milestone 3: Uploading the Retail Dataset into PostgreSQL

Group members: Pratham Patel (202618057), Rohan Deshpande (202618058), Hiteshsinh Rathod (202618040)

Server: PostgreSQL at `10.100.71.21`, database `202618057`

In this milestone, each of the 12 CSV files is read into a Pandas DataFrame and loaded into the PostgreSQL tables created from the Milestone 2 DDL. The connection is made with `sqlalchemy`, and tables are loaded parents first so every foreign key resolves.

1. Imports

In [1]:
```
import os
import time
import getpass

import pandas as pd
from sqlalchemy import create_engine, text, inspect

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 150)
print("pandas", pd.__version__)
```

```
pandas 3.0.5
```

2. Connection Settings

The password is entered at run time and is not stored in the notebook.

In [2]:
```
DB_HOST = "10.100.71.21"
DB_PORT = 5432
DB_NAME = input("Database name: ").strip()
DB_USER = input("User name: ").strip()
DB_PASS = getpass.getpass("Password: ")

# Folder that contains the 12 CSV files
DATA_DIR = "data"

# Schema where the DDL was run ("public" unless you created your own schema)
SCHEMA = "public"
```

2.1 Connect to the PostgreSQL Server

In [3]:
```
from sqlalchemy.engine import URL

url = URL.create(
    "postgresql+psycopg2",
    username=DB_USER,
    password=DB_PASS,
    host=DB_HOST,
    port=DB_PORT,
    database=DB_NAME,
)
engine = create_engine(url, pool_pre_ping=True)

with engine.connect() as conn:
    print(conn.execute(text("SELECT version()")).scalar())
```

```
PostgreSQL 10.20 on x86_64-pc-linux-gnu, compiled by gcc (GCC) 4.8.5 20150623 (Red Hat 4.8.5-44), 64-bit
```

3. Verify the Target Tables

Check that all 12 tables created by the Milestone 2 DDL exist in the database.

In [4]:
```
LOAD_ORDER = [            # parents first, children after
    "categories", "suppliers", "stores", "promotions", "customers",
    "employees", "products", "orders",
    "order_items", "payments", "shipments", "returns",
]

existing = set(inspect(engine).get_table_names(schema=SCHEMA))
missing = [t for t in LOAD_ORDER if t not in existing]
print("Tables found  :", sorted(existing))
print("Tables missing:", missing if missing else "none")
assert not missing, "Run the Milestone 2 DDL first."
```

```
Tables found  : ['categories', 'customers', 'department', 'employees', 'order_items', 'orders', 'payments', 'products', 'program', 'promotions',
'returns', 'shipments', 'stores', 'student', 'suppliers']
Tables missing: none
```

4. Inspect the CSV Files

Read every CSV file and compare its columns with the columns of the matching database table.

In [5]:
```
db_columns = {
    t: [c["name"] for c in inspect(engine).get_columns(t, schema=SCHEMA)]
    for t in LOAD_ORDER
}

summary = []
for t in LOAD_ORDER:
    path = os.path.join(DATA_DIR, f"{t}.csv")
    df = pd.read_csv(path)
    df.columns = [c.strip().lower() for c in df.columns]
    summary.append({
        "table": t,
        "csv_rows": len(df),
        "csv_columns": list(df.columns),
        "db_columns_ok": set(db_columns[t]).issubset(df.columns),
    })

pd.DataFrame(summary)
```

	table	csv_rows	csv_columns	db_columns_ok
0	categories	30	[category_id, category_name]	True
1	suppliers	200	[supplier_id, country]	True
2	stores	100	[store_id, city]	True
3	promotions	50	[promotion_id, discount]	True
4	customers	50000	[customer_id, city, signup_date]	True
5	employees	1000	[employee_id, store_id, salary]	True
6	products	10000	[product_id, category_id, supplier_id, price]	True
7	orders	300000	[order_id, customer_id, store_id, order_date, ...	True
8	order_items	600000	[order_item_id, order_id, product_id, qty, price]	True
9	payments	300000	[payment_id, order_id, amount]	True
10	shipments	300000	[shipment_id, order_id, status]	True
11	returns	30000	[return_id, order_item_id, refund]	True

4.1 Sample Rows from the CSV Files

In [6]:
```
print(pd.read_csv(os.path.join(DATA_DIR, "orders.csv")).head())
print()
print(pd.read_csv(os.path.join(DATA_DIR, "order_items.csv")).head())
```

```
   order_id  customer_id  store_id  order_date  promotion_id
0         1        45308        33  2021-08-26            24
1         2        10070        81  2022-03-19             3
2         3        43308        17  2021-01-21            25
3         4        47997        85  2021-01-16            48
4         5        36546        81  2022-09-14            33

   order_item_id  order_id  product_id  qty  price
0              1    145042         472    3    176
1              2    110932        1666    2   1034
2              3    269799        8616    4   2290
3              4    298741        9909    3   1555
4              5    105218        1179    3    936
```

5. Loading Configuration

Date columns, chunk size and the load order (parents before children).

In [7]:
```
DATE_COLUMNS = {
    "customers": ["signup_date"],
    "orders": ["order_date"],
}
CHUNK = 5000   # 5000 rows x 5 columns stays below the 65,535 parameter limit of 'multi' inserts

def load_table(table):
    path = os.path.join(DATA_DIR, f"{table}.csv")
    df = pd.read_csv(path)
    df.columns = [c.strip().lower() for c in df.columns]

    # keep only the table's columns, in the table's order
    df = df[db_columns[table]]

    # IDs and counts -> nullable integer
    for col in df.columns:
        if col.endswith("_id") or col in ("qty", "discount"):
            df[col] = df[col].astype("Int64")

    # dates -> python date
    for col in DATE_COLUMNS.get(table, []):
        df[col] = pd.to_datetime(df[col]).dt.date

    start = time.time()
    df.to_sql(
        name=table,
        con=engine,
        schema=SCHEMA,
        if_exists="append",     # required by the brief: keep our DDL table
        index=False,
        chunksize=CHUNK,
        method="multi",
    )
    return len(df), time.time() - start
```

5.1 Reset Option

Set `RESET_TABLES = True` only to empty the tables before a fresh load.

In [8]:
```
RESET_TABLES = False   # set to True to empty all tables first

if RESET_TABLES:
    with engine.begin() as conn:
        conn.execute(text(
            "TRUNCATE TABLE " + ", ".join(f"{SCHEMA}.{t}" for t in LOAD_ORDER) + " CASCADE"
        ))
    print("All tables truncated.")
else:
    print("Skipped.")
```

```
Skipped.
```

5.2 Loader Function

Reads a CSV into a DataFrame, prepares ID and date columns, and appends the rows to the table through the SQLAlchemy connection.

In [9]:
```
import io

def load_table(table):
    path = os.path.join(DATA_DIR, f"{table}.csv")
    df = pd.read_csv(path)
    df.columns = [c.strip().lower() for c in df.columns]
    df = df[db_columns[table]]

    for col in df.columns:
        if col.endswith("_id") or col in ("qty", "discount"):
            df[col] = df[col].astype("Int64")
    for col in DATE_COLUMNS.get(table, []):
        df[col] = pd.to_datetime(df[col]).dt.date

    buf = io.StringIO()
    df.to_csv(buf, index=False, header=False)
    buf.seek(0)

    start = time.time()
    raw = engine.raw_connection()
    try:
        cur = raw.cursor()
        cur.copy_expert(
            f"COPY {SCHEMA}.{table} ({', '.join(df.columns)}) FROM STDIN WITH (FORMAT csv)",
            buf,
        )
        raw.commit()
    except Exception:
        raw.rollback()
        raise
    finally:
        raw.close()
    return len(df), time.time() - start
```

6. Load All 12 Tables

The tables are emptied first, then loaded one by one. After each table the row count is read back from the database.

In [10]:
```
# 1. Empty all tables (safe, keeps the structure)
with engine.begin() as conn:
    conn.execute(text(
        "TRUNCATE TABLE " + ", ".join(f"{SCHEMA}.{t}" for t in LOAD_ORDER) + " CASCADE"
    ))
print("Tables emptied.\n")

# 2. Load every table and confirm the count straight away
for t in LOAD_ORDER:
    n, secs = load_table(t)
    with engine.connect() as conn:
        in_db = conn.execute(text(f"SELECT COUNT(*) FROM {SCHEMA}.{t}")).scalar()
    print(f"{t:<12} read {n:>9,}   in database {in_db:>9,}   ({secs:.1f} s)")
```

```
Tables emptied.

categories   read        30   in database        30   (0.0 s)
suppliers    read       200   in database       200   (0.1 s)
stores       read       100   in database       100   (0.0 s)
promotions   read        50   in database        50   (0.0 s)
customers    read    50,000   in database    50,000   (0.2 s)
employees    read     1,000   in database     1,000   (0.1 s)
products     read    10,000   in database    10,000   (0.3 s)
orders       read   300,000   in database   300,000   (10.2 s)
order_items  read   600,000   in database   600,000   (14.5 s)
payments     read   300,000   in database   300,000   (4.8 s)
shipments    read   300,000   in database   300,000   (5.1 s)
returns      read    30,000   in database    30,000   (1.0 s)
```

7. Close the Connection

In [11]:
```
engine.dispose()
print("Connection closed.")
```

```
Connection closed.
```

8. Summary

- **Connection:** The notebook connected to PostgreSQL 10.20 at `10.100.71.21` (database `202618057`) through SQLAlchemy. The password is entered at run time and is not stored.
- **Tables:** All 12 tables from the Milestone 2 DDL were found in the database. Other tables in the database (`department`, `program`, `student`) were not touched.
- **CSV files:** The 12 CSV files were read with `pd.read_csv`. For every file, the columns matched the columns of its database table.
- **Loading:** The tables were emptied first and then loaded parents first, so every foreign key found its parent row. The rows were loaded with PostgreSQL's `COPY` command through the SQLAlchemy connection. `to_sql` could not be used on our machine because the installed pandas (3.0.5) and SQLAlchemy (2.0.31) versions are not compatible.
- **Result:** After each table was loaded, its row count was read back from the database. Every table's count equals the number of rows read from its CSV file, and **1,591,380 rows** were loaded in total.

Table	Rows loaded	Table	Rows loaded
categories	30	orders	300,000
suppliers	200	order_items	600,000
stores	100	payments	300,000
promotions	50	shipments	300,000
customers	50,000	returns	30,000
employees	1,000	products	10,000